Step 2 — The Planner and the RAG Search — Solution

> Back to the step · Hint 1 · Hint 2

Story: the planner (a structured LLM call) and the RAG search (a document store + a `search` the executor calls). **If not:** without the planner there's no "plan first"; without a cited search, findings have nothing to point back to.

The planner (plan-first)

```
# src/planner.py
# Break a broad question into a few focused sub-questions - Doc07's plan-first,
# as one structured call.
from langchain_core.prompts import ChatPromptTemplate

from src.schemas import SubQuestions

PLAN_PROMPT = (
    "Break the research question below into 3-5 focused sub-questions that, "
    "answered together, cover it. Return just the sub-questions.\n\n"
    "Research question: {question}"
)

def make_plan(model, question):
    prompt = ChatPromptTemplate.from_template(PLAN_PROMPT)
    chain = prompt | model.with_structured_output(SubQuestions)
    return chain.invoke({"question": question})
```

The source store

```
# src/rag/source_store.py
# Load the source documents, keep each file's name as metadata (so findings can
# cite it), chunk, embed, and store in Chroma.
from langchain_community.document_loaders import DirectoryLoader, TextLoader
from langchain_openai import OpenAIEmbeddings
from langchain_text_splitters import RecursiveCharacterTextSplitter
```

```
import chromadb

COLLECTION_NAME = "sources"
EMBEDDINGS = OpenAIEmbeddings(model="text-embedding-3-small")

def build_source_store(sources_dir="data/sources"):
    loader = DirectoryLoader(sources_dir, glob="**/*.md",
                             loader_cls=TextLoader)
    documents = loader.load()
    splitter = RecursiveCharacterTextSplitter(chunk_size=400, chunk_overlap=40)
    chunks = splitter.split_documents(documents)

    client = chromadb.Client()
    try:
        client.delete_collection(COLLECTION_NAME)
    except Exception:
        pass
    collection = client.create_collection(COLLECTION_NAME)

    ids = []
    texts = []
    metadatas = []
    for index in range(len(chunks)):
        ids.append(f"chunk-{index}")
        texts.append(chunks[index].page_content)
        source = chunks[index].metadata.get("source", "unknown")
        # keep just the file name, not the full path
        metadatas.append({"source": source.split("/")[-1]})
    collection.add(ids=ids, documents=texts, metadatas=metadatas,
                   embeddings=EMBEDDINGS.embed_documents(texts))
    return collection
```

The search function

```
# src/rag/search.py
# make_search returns a search(q) -> (text, source) closure over the collection.
# The executor calls this once per sub-question - it doesn't know about Chroma.
from src.rag.source_store import EMBEDDINGS

def make_search(collection):
    def search(query):
        if collection.count() == 0:
            return "No sources available.", "none"
        results = collection.query(
```

```
            query_embeddings=[EMBEDDINGS.embed_query(query)], n_results=1,
        )
        documents = results["documents"][0]
        metadatas = results["metadatas"][0]
        if not documents:
            return "No information found.", "none"
        return documents[0], metadatas[0].get("source", "unknown")

    return search
```

Why these two pieces

- **The planner is the whole "plan-first" idea.** One structured call turns a broad question into a written list of sub-questions — a plan you can read before anything runs, which is exactly what makes a research agent reviewable instead of a black box.
- **The search is a closure, keyed on your documents' filenames.** `make_search(collection)` hands the executor a `search(q) -> (text, source)` that it calls once per sub-question. The executor never imports Chroma — it just calls the function, which is why Step 1 could test it with a fake one.
- **The source metadata is the citation.** Keeping each chunk's filename in `metadata["source"]` is what lets a finding point back to `rag_overview.md` — no filename, no citation.

The live run — real plan, real search, real citations — is Step 3.